

Kulak Burun Boğaz Kliniği

Veri tabanı Projesi

Hazırlayanlar:

Ömer Faruk Bakırcı	2211502042
Baran Kanat	2311502274
Gürhan Kaya	2311502277
<i>Yakup Çakmak</i>	<i>2211502077</i>

Katılımcıların Görev Dağılımı:

Ömer Faruk Bakırcı: Hasta, Randevu, Muayene, Tanı, Fatura tablolarının oluşturulması. ER diyagramının hazırlanması. Normalizasyon kontrolü. SQL sorgu oluşturma

Gürhan Kaya: Hasta, Tahlil, Doktor, Personel tablolarının oluşturulması. İlişkisel şemanın oluşturulması. Raporun hazırlanması. SQL sorgu oluşturma

Baran Kanat: Hasta, Reçete, Reçete_ilaç, Kullanıcı, İlaç, Çalışma_takvimi tablolarının oluşturulması. Trigger eklenmesi. SQL sorgu oluşturma

Yakup Çakmak: SQL sorgu oluşturma. Açıkamaların hazırlanması

1. Proje Hakkında Genel Açıklama:

Bu proje, Kulak Burun Boğaz (KBB) alanında hizmet veren bir kliniğin hasta kayıtları, doktorlar, randevular, muayeneler, tanılar, tedaviler, reçeteler, ilaçlar ve tahliller gibi temel klinik işleyişleri dijital ortamda takip edebileceği bir veri tabanı sistemi oluşturmayı amaçlamaktadır. Proje, Veri tabanı Yönetimi dersi kapsamında, gerçek hayattaki sağlık bilgi sistemlerinin küçük ölçekli bir simülasyonu olarak tasarlanmıştır. Sistem, klinik operasyonların daha düzenli, güvenilir ve hızlı yürütülmesini sağlamayı hedefler. Projenin uygulama aşamasında, ilişkisel veri tabanı yönetim sistemi olarak Oracle Live SQL kullandık.

Bilgi sisteminin temel işlevleri şu şekilde sıralanabilir:

- Bilgi sistemi veri tabanına yeni hasta kaydı ekleme
- Bilgi sistemi veri tabanında yeni randevu oluşturma
- Bilgi sistemi veri tabanına muayene kaydı ekleme
- Bilgi sistemi veri tabanında doktor bilgisi güncelleme

1.1 Projenin Amacı:

- Hasta bilgilerinin güvenli ve merkezi biçimde saklanması
- Randevu sisteminin dijitalleştirilmesi
- Doktor, personel ve muayene süreçlerinin takibi
- Tedavi planlarının ve reçetelerin sistem üzerinden oluşturulması
- Laboratuvar tahlillerinin ve sonuçlarının kaydedilmesi
- Klinik envanter ve ilaç stoklarının takibi
- Fatura ve ödeme süreçlerinin kayıt altına alınması

1.2 Projenin Kapsadığı Alanlar:

- Hasta yönetimi
- Randevu yönetimi
- Doktor ve personel takibi
- Muayene ve tanı süreçleri
- ilaç uygulamaları
- Reçete yönetimi
- Laboratuvar testleri
- Faturalama işlemleri

1.3 Kullanılan Teknolojiler:

- Oracle Live SQL platformu üzerinden ilişkisel veri tabanı tasarımı ve sorgulama
- app.diagrams.net sitesi ile kavramsal diyagramı görselleştirilmesi
- dbdiagram.io sitesi ile ilişkisel modelin görselleştirilmesi
- chatgpt.com sitesi ile verilerin oluşturulması

1.4 Çözüm Getirilen Problemler:

- Manuel (kağıt kalem) sistemlerin sebep olduğu veri kaybı ve karışıklıkların önlenmesi
- Aynı hastanın farklı zamanlarda girilen kayıtlarının bütüncül olarak izlenememesi
- Yoğun randevu trafiğinde hasta-doktor planlaması hatalarının önlenmesi
- Klinik içi stok takibinin eksik yapılması
- İlaç ve reçete süreçlerinin kontrolsüz ilerlemesi
- Laboratuvar test sonuçlarının takibinde yaşanan bilgi kayıplarının engellenmesi

2. Senaryo – Varlıklar, Öznitelikler, İlişkiler

2.1 Varlıklar ve Öznitelikleri:

Hasta (HastaID, Ad, Soyad, TCKN, DoğumTarihi, Cinsiyet, Telefon, Eposta, Adres, BoyCM, KiloKG, SigortaTuru, Alerjiler, KronikHastalıklar, KanGrubu)

Doktor (DoktorID, Ad, Soyad, Branş, DiplomaNo, Unvan, OdaID, Telefon, Eposta, Maaş)

Personel (PersonelID, Ad, Soyad, Görev, Departman, Telefon, Eposta, Maaş)

Randevu (RandevuID, HastaID, DoktorID, TarihSaat, Durum)

Muayene (MuayeneID, RandevuID, DoktorID, Bulgular, Notlar, Epikriz, MuayeneTarihi)

Tanı (TaniID, MuayeneID, ICD10Kodu, Açıklama, TaniTarihi)

İlaç (IlacID, IlacAdi, EtkenMadde, Dozaj, Form, YanEtkiler)

Reçete (ReceteID, HastaID, DoktorID, MuayeneID, Tarih, Notlar)

Reçete-İlaç (ReceteID, IlacID, Dozaj, Sıklık, GünSayisi, Miktar)

Tahlil (TestID, HastaID, DoktorID, MuayeneID, TestTuru, Sonuc, Durum, TestTarihi)

Fatura (FaturaID, HastaID, RandevuID, HizmetAciklamasi, Tutar, SigortaKapsami, OdemeDurumu)

Kullanıcı (KullaniciID, DoktorID, KullaniciAdi, Sifre, YetkiSeviyesi, PersonelID)

Çalışma Takvimi (TakvimID, DoktorID, Gün, BaşlangicSaat, BitisSaat)

2.2 Varlıklar Arası İlişkiler:

- Hasta – Randevu: (1:N) → Bir hasta birden fazla randevu alabilir.
- Doktor – Randevu: (1:N) → Bir doktor birden fazla randevu verebilir.
- Randevu – Muayene: (1:1) → Her randevuya karşılık bir muayene vardır.
- Muayene – Tanı: (1:N) → Bir muayenede birden fazla tanı olabilir.
- Muayene – Reçete: (1:1) → Her muayeneye bir reçete yazılabilir.
- Reçete – İlaç: (M:N) → Bir reçetede birden fazla ilaç olabilir.
- Hasta – Tahlil: (1:N) → Her hastanın birden fazla tahlili olabilir.
- Muayene – Tahlil: (1:N) → Her muayeneye birden fazla tahlil bağlanabilir.
- Hasta – Fatura: (1:N) → Her hastaya birden fazla fatura kesilebilir.
- Randevu – Fatura: (1:1) → Her randevunun bir faturası olabilir.
- Doktor – Çalışma Takvimi: (1:N) → Her doktorun haftalık takvimi olabilir.
- Doktor – Kullanıcı: (1:1) → Her doktor sisteme bir kullanıcı olarak bağlanabilir.
- Personel – Kullanıcı: (1:1) → Her personelin kullanıcı hesabı olabilir.

3. Varlık-İlişki Diyagramı (ER)

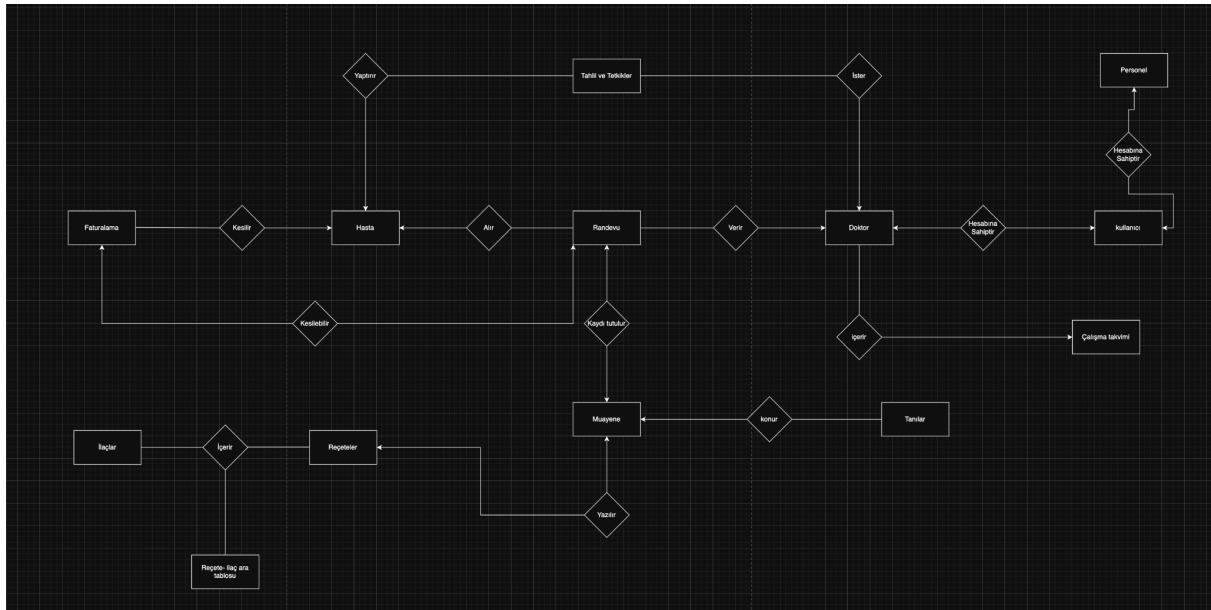
3.1 Varlıklar:

Güçlü Varlıklar:

- Hasta
- Doktor
- Randevu
- Muayene

Zayıf Varlıklar:

- Tanı
- Reçete
- Recete_Ilac
- Çalışma Takvimi
- Personel
- İlaç
- Tahlil
- Fatura
- Kullanıcı



4. ER'den İlişkisel Şemaya Dönüşüm

1. Adım: Güçlü Varlıkların Eşlenmesi

Her güçlü varlık, doğrudan bir tabloya dönüştürülür.

Örnek Tablolar: Hasta, Doktor, Randevu

2. Adım: Zayıf Varlıkların Eşlenmesi

Zayıf varlıklar, bağlı oldukları güçlü varlığın birincil anahtarını (PK) yabancı anahtar (FK) olarak alır.

Örnek Tablo: Tanılar (TanıID, MuayeneID)

3. Adım: 1:1 İlişki Türlerinin Eşlenmesi

İki varlık arasındaki birebir ilişkide, bir varlık diğerinin birincil anahtarını FK olarak alır.

Örnek: Muayene tablosu, RandevuID'yi FK olarak içerir.

4. Adım: 1:N İlişki Türlerinin Eşlenmesi

Birçoktan bire (1:N) ilişkilerde, "N" tarafındaki tablo, "1" tarafın PK'sini FK olarak içerir.

Örnek: Randevu tablosu, HastaID'yi FK olarak içerir.

5. Adım: M:N İlişki Türlerinin Eşlenmesi

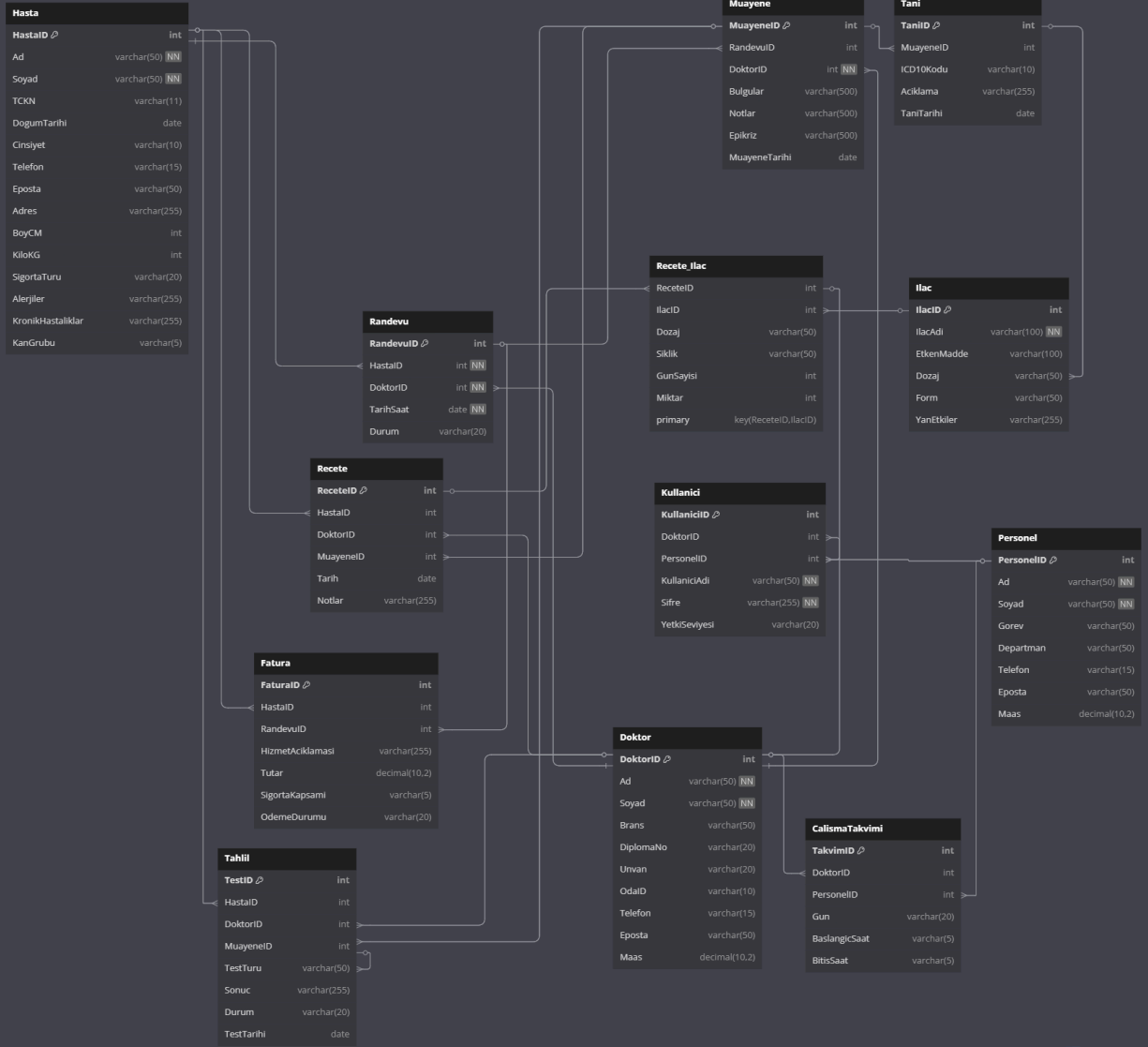
M:N ilişkiler için bir ara tablo oluşturulur ve her iki varlığın PK'si burada FK olarak tanımlanır.

Örnek Ara Tablo: Recete_Ilac (ReceteID, IlacID)

6. Adım: N-ary (3+ Varlıklı) İlişki Türlerinin Eşlenmesi

Üç veya daha fazla varlıkla kurulan ilişkiler ayrı bir tablo ile temsil edilir, bu tablo her varlığın PK'sini FK olarak içerir.

Örnek Tablolar: Tahlil (HastaID, DoktorID, MuayeneID), Fatura (HastaID, RandevuID)



5. Aşama: Normalizasyon Kontrolü

Bu aşamada veritabanı tablolarının 1NF, 2NF ve 3NF kurallarına uygunluğu kontrol edilmiştir. Bu kontrol ile veri tekrarlarının önlenmesi, veri tutarlılığının sağlanması ve veri güncelleme işlemlerinde oluşabilecek hataların azaltılması hedeflenmiştir.

5.1 1NF (Birinci Normal Form):

- Her hücrede tek bir (atomik) veri bulunmalıdır.
 - Her sütun yalnızca bir veri türü içermelidir.
 - Tablodaki her satır benzersiz olmalıdır.
 - Çoklu değer içeren alanlar bulunmamalıdır (örn. "Telefon1, Telefon2" gibi virgülle ayrılmış veri kullanılmaz).
- Tüm tablolarımızda yukarıdaki kurallara uyduğu için 1NF şartı sağlanmıştır.

Örnek:

Tablo: Hasta

Kolonlar: Ad, Soyad, TCKN, DogumTarihi, Telefon, Adres

Bu tablo ve kolonlar 1NF kurallarına uymaktadır çünkü:

- Her sütun **atomik veri** içerir (örneğin Telefon sadece bir numara içerir),
- Her sütun **tek bir veri türüne** sahiptir (örn. DogumTarihi DATE türünde),
- HastaID birincil anahtar olarak kullanıldığı için **her satır benzersizdir**.

5.2 2NF (İkinci Normal Form):

- Öncelikle 1NF sağlanmış olmalıdır.
 - Birincil anahtarın parçasına bağımlı olan kısmi bağımlılıklar kaldırılmalıdır.
- Tüm tablolarımızda yukarıdaki kurallara uyduğu için 2NF şartı sağlanmıştır.

örnek:

Tablo: Randevu

Kolonlar: RandevuID (birincil anahtar), HastaID, DoktorID, TarihSaat, Durum

Bu tablo 2NF kurallarına uygundur çünkü:

- Birincil anahtar tek sütundan (RandevuID) oluşmaktadır,
- Diğer tüm sütunlar (HastaID, DoktorID, TarihSaat, Durum) doğrudan bu anahtara tam bağımlıdır,
- Bileşik anahtar olmadığından kısmi bağımlılık da yoktur.

5.3 3NF (Üçüncü Normal Form):

- Öncelikle 2NF sağlanmış olmalıdır.
- Tablolarda öznelikler arasında dolaylı bağımlılık olmamalıdır. Tüm öznelikler doğrudan birincil anahtar ile ilişkili olmalıdır.

örnek:

Tablo: Tani

Sütunlar: TaniID (PK), MuayeneID, ICD10Kodu, Aciklama

Bu tablo 3NF'e uygundur çünkü:

- Tüm sütunlar doğrudan TaniID'ye bağlıdır,
- Aralarında transitif bağımlılık yoktur,
- ICD10Kodu gibi sütunlar başka sütunlara bağlı değildir.

Genel Değerlendirme:

Normal Form	Uygunluk	Açıklama
1NF	✓	Tüm tablolar tekil hücreler içeriyor.
2NF	✓	Birleşik PK kullanılan tablolarda bağımlılık doğru kurulmuş.
3NF	✓	Tablolar arası transitive bağımlılık yok.

6. Aşama: Sorgular

Örnek Sorgular 6.1:

1.1 Tüm hastaların ad, soyad ve kan grubu bilgilerini listele

```
SELECT Ad, Soyad, KanGrubu FROM Hasta;
```

1.2 Sigorta türü "SGK" olan hastaları listele

```
SELECT Ad, Soyad, SigortaTuru FROM Hasta WHERE SigortaTuru = 'SGK';
```

1.3 Randevusu "Tamamlandı" olan hastaların adlarını ve randevu tarihlerini getir

```
SELECT H.Ad, H.Soyad, R.TarihSaat
```

```
FROM Hasta H
```

```
JOIN Randevu R ON H.HastaID = R.HastaID
```

```
WHERE R.Durum = 'Tamamlandı';
```

1.4 Her doktora ait toplam randevu sayısını göster

```
SELECT D.Ad, D.Soyad, COUNT(*) AS RandevuSayisi
```

```
FROM Doktor D
```

```
JOIN Randevu R ON D.DoktorID = R.DoktorID
```

```
GROUP BY D.Ad, D.Soyad;
```

1.5 En çok randevu verilen doktorun adını ve toplam randevu sayısını getir

```
SELECT Ad, Soyad, RandevuSayisi FROM (  
  
    SELECT D.Ad, D.Soyad, COUNT(*) AS RandevuSayisi  
  
    FROM Doktor D  
  
    JOIN Randevu R ON D.DoktorID = R.DoktorID  
  
    GROUP BY D.Ad, D.Soyad  
  
    ORDER BY RandevuSayisi DESC  
  
)  
  
WHERE ROWNUM = 1;
```

Örnek Sorgular 6.2:

2.1 SGK kapsamındaki faturaları listele

```
SELECT * FROM Fatura WHERE SigortaKapsami = 'Evet';
```

2.2 Muayene teşhislerini ve açıklamalarını listele

```
SELECT T.ICD10Kodu, T.Aciklama, M.MuayeneTarihi  
  
FROM Tani T  
  
JOIN Muayene M ON T.MuayeneID = M.MuayeneID;
```

2.3 İlaçların formuna göre kaç farklı ilaç olduğunu say

```
SELECT Form, COUNT(*) AS IlacSayisi  
  
FROM Ilac  
  
GROUP BY Form;
```

2.4 Reçete ve ilaç ilişkilerinde hangi ilaç kaç kere reçetelenmiş

```
SELECT I.IlacAdi, COUNT(*) AS ReçeteSayisi  
FROM Recete_Ilac RI  
JOIN Ilac I ON RI.IlacID = I.IlacID  
GROUP BY I.IlacAdi;
```

2.5 En çok ilaç yazan doktorun adını ve yazdığı toplam ilaç sayısını getir

```
SELECT D.Ad, D.Soyad, COUNT(*) AS IlacYazmaSayisi FROM Recete R  
JOIN Recete_Ilac RI ON R.ReceteID = RI.ReceteID  
JOIN Doktor D ON R.DoktorID = D.DoktorID  
GROUP BY D.Ad, D.Soyad  
ORDER BY IlacYazmaSayisi DESC FETCH FIRST 1 ROWS ONLY;
```

Örnek Sorgular 6.3:

3.1 "Kadın" cinsiyetine sahip hastaların isim ve doğum tarihlerini getir

```
SELECT Ad, Soyad, DogumTarihi FROM Hasta WHERE Cinsiyet = 'Kadın';
```

3.2 Her hasta için kaç reçetesi olduğunu göster

```
SELECT H.Ad, H.Soyad, COUNT(*) AS ReceteSayisi  
FROM Hasta H  
JOIN Recete R ON H.HastaID = R.HastaID  
GROUP BY H.Ad, H.Soyad;
```

3.3 En pahalı 3 hizmeti göster (fatura)

```
SELECT HizmetAciklamasi, Tutar
```

```
FROM Fatura
```

```
ORDER BY Tutar DESC FETCH FIRST 3 ROWS ONLY;
```

3.4 "Alerji Paneli" testi yaptıran hastaları ve test sonucunu getir

```
SELECT H.Ad, H.Soyad, T.Sonuc
```

```
FROM Tahlil T
```

```
JOIN Hasta H ON T.HastaID = H.HastaID
```

```
WHERE T.TestTuru = 'Alerji Paneli';
```

3.5 En az bir reçetesi olan ve en az bir tahlil yaptırmış hastaları getir

```
SELECT DISTINCT H.Ad, H.Soyad
```

```
FROM Hasta H
```

```
WHERE EXISTS (
```

```
    SELECT 1 FROM Recete R WHERE R.HastaID = H.HastaID
```

```
)
```

```
AND EXISTS (
```

```
    SELECT 1 FROM Tahlil T WHERE T.HastaID = H.HastaID
```

```
);
```

Örnek Sorgular 6.4:

4.1 Erkek hastaları boylarına göre büyükten küçüğe sırala

```
SELECT Ad, Soyad, BoyCM FROM Hasta WHERE Cinsiyet = 'Erkek' ORDER BY BoyCM  
DESC;
```

4.2 Çalışma takviminde cuma günü çalışan doktorları getir

```
SELECT D.Ad, D.Soyad  
  
FROM Doktor D  
  
JOIN CalismaTakvimi C ON D.DoktorID = C.DoktorID  
  
WHERE C.Gun = 'Cuma';
```

4.3 Randevusu "Bekliyor" durumunda olan hastaları getir

```
SELECT H.Ad, H.Soyad, R.TarihSaat  
  
FROM Randevu R  
  
JOIN Hasta H ON R.HastaID = H.HastaID  
  
WHERE R.Durum = 'Bekliyor';
```

4.4 Aynı muayeneye birden fazla tanı konulan muayeneleri göster

```
SELECT MuayeneID, COUNT(*) AS TaniSayisi  
  
FROM Tani  
  
GROUP BY MuayeneID  
  
HAVING COUNT(*) > 1;
```

4.5 Hiç reçetesi olmayan hastaları listele

SELECT H.Ad, H.Soyad

FROM Hasta H

WHERE NOT EXISTS (

SELECT 1 FROM Recete R WHERE R.HastaID = H.HastaID);

1.5 Numaralı sorgunun ekran görüntüsü

	AD	SOYAD	RANDEVUSAYISI
1	Zeynep	Demirtaş	2

2.5 Numaralı sorgunun ekran görüntüsü

	AD	SOYAD	ILACYAZMASAYISI
1	Zeynep	Demirtaş	2

3.5 Numaralı sorgunun ekran görüntüsü

	AD	SOYAD
1	Ahmet	Yılmaz
2	Zeynep	Kara
3	Elif	Şahin
4	Mehmet	Demir

4.5 Numaralı sorgunun ekran görüntüsü

	AD	SOYAD
1	Burcu	Öztürk
2	Nazlı	Yılmaz
3	Cihan	Karaoğlu
4	Emre	Koç
5	Volkan	Yavuz
6	Ümit	Taş
7	Selin	Yıldırım
8	Ayşe	Polat
9	Deniz	Aydın
10	Seda	Güneş